บทที่ 10: การประมวลผลแบบแบตช์ (**Batch Processing)**

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 15–18 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ **Designing Data-Intensive Applications** บทที่สิบลื่นไหลเข้าสู่ภาคที่สาม ซึ่งเป็นภาคสุดท้ายของหนังสือเล่มนี้ ว่าด้วยเรื่อง ข้อมูลที่ถูกสร้างขึ้นจากข้อมูลอื่น (**Derived Data)** และเปิดฉากด้วยบท การประมวลผลแบบแบตช์ (**Batch Processing)**

ในบทที่ผ่านมา เราพูดถึงระบบออนไลน์ที่ผู้ใช้รอคำตอบแบบเรียลไทม์ และระบบสตรีมที่ข้อมูลไหลมาไม่หยุด แต่ในโลกความเป็นจริง งานคำนวณส่วนใหญ่ที่มีขนาดใหญ่ระดับเพตาไบต์ เช่น การคำนวณรายงานประจำปี การสร้างดัชนีเครื่องมือค้นหา หรือการฝึกโมเดลปัญญาประดิษฐ์ ไม่จำเป็นต้องเสี้ยววินาที งานเหล่านี้มักรวบรวมข้อมูลก้อนโตแล้วสั่งรันครั้งเดียวจบ ซึ่งเรียกว่า **Batch Processing**

บทนี้จะพาไปย้อนดูรากฐานของระบบประมวลผลข้อมูล ตั้งแต่เครื่องมือคลาสสิกของยูนิกซ์ ระบบกระจายอย่าง **MapReduce** จนถึงเครื่องมือยุคใหม่ครับ

ตอนที่ 2: ปรัชญาจากเครื่องมือของ **Unix**

ผู้เขียนพาเราย้อนกลับไปดูต้นกำเนิดของแนวคิดการประมวลผลข้อมูล นั่นคือ **Unix Command-Line Tools** เช่น **cat, grep, sort, awk,** และ **sed**

คำสั่งเหล่านี้เรียบง่ายแต่ทรงพลังมาก เพราะถูกออกแบบตามปรัชญาสำคัญ:

- ทำหน้าที่อย่างเดียวแต่ทำให้อดีต: แต่ละคำสั่งทำภารกิจเฉพาะตัว
- เชื่อมต่อด้วยท่อ (**Pipes):** เอาผลลัพธ์ของคำสั่งแรกส่งเป็นอินพุตให้คำสั่งถัดไปผ่านเครื่องหมาย **| (Pipe)**
- ข้อมูลต้นทางห้ามแก้ (**Immutable Inputs):** ข้อมูลดิบที่ถูกอ่านเข้ามาจะไม่ถูกแก้ไขทำลาย แต่จะถูกแปรรูปส่งต่อเป็นข้อมูลชุดใหม่ไปเรื่อย ๆ

แม้ปรัชญาของยูนิกซ์จะยอดเยี่ยม แต่พอดึงข้อมูลขึ้นมาทำบนเครื่องเดียวสเกลใหญ่ ๆ มันก็มีข้อจำกัดเรื่องฮาร์ดแวร์ จึงเป็นที่มาของการขยายแนวคิดนี้ไปยังระบบกระจาย นั่นคือ **MapReduce**

ตอนที่ 3: MapReduce และระบบแฟ้มข้อมูลแบบกระจาย

MapReduce ที่พัฒนาโดย **Google** และต่อมาอยู่ในระบบ **Hadoop** คือกรอบงาน (**Framework)** ที่จำลองแนวคิดของ **Unix Pipes** ออกมาวิ่งบนเครื่องนับพันเครื่องพร้อมกัน

การทำงานของ **MapReduce** มีหัวใจหลักอยู่ **2** ฟังก์ชันที่เราต้องเขียนโค้ดเอง:

1. **Mapper:** ฟังก์ชันที่วิ่งไปอ่านข้อมูลดิบทีละเรเบียน แล้วคัดเลือกหรือแยกย่อยออกมาเป็นคู่คีย์-ค่า (**Key-Value)** แต่ละเรเบียนทำงานอิสระต่อกัน ไม่ยุ่งเกี่ยวกับใคร

2. **Reducer:** ฟังก์ชันที่จะรวบรวมข้อมูลทั้งหมดที่มี "คีย์เดียวกัน" ซึ่งถูกจัดกลุ่มและเรียงลำดับมาให้เรียบร้อยแล้วจากกระบวนการเบื้องหลัง (**Shuffle)** เพื่อนำมาคำนวณสรุปผล เช่น รวมยอด หรือนับจำนวน

ส่วนตัวกลางที่คอยเก็บไฟล์ข้อมูลดิบและผลลัพธ์คั่นกลางระหว่างงาน คือระบบแฟ้มข้อมูลแบบกระจาย เช่น **HDFS (Hadoop Distributed File System)** ซึ่งใช้หลักการเขียนข้อมูลครั้งเดียวแล้วอ่านซ้ำได้เรื่อย ๆ และทำสำเนาหลายเครื่องเพื่อกันข้อมูลหาย

ตอนที่ **4:** ข้อดีและจุดอ่อนของ **MapReduce**

จุดเด่น:

- ทนทานต่อความพัง (**Fault Tolerance):** เนื่องจาก **MapReduce** เขียนผลลัพธ์คั่นกลางลงดิสก์ทุกขั้นตอน ถ้าเครื่องไหนพังกลางทางระหว่างรันงาน ระบบจะแค่สั่งรันงานของเครื่องนั้นใหม่บนเครื่องอื่นโดยที่งานส่วนอื่นไม่พังตาม
- จัดการขุดคุ้ยข้อมูลมหาศาลได้ดี: เหมาะกับการทำ **ETL (Extract, Transform, Load)** เพื่อแปลงข้อมูลดิบในคลังข้อมูลให้อยู่ในรูปแบบพร้อมวิเคราะห์

จุดอ่อน:

- ช้าเพราะเขียนลงดิสก์: ทุกขั้นตอนของการต่อ **Job (**เช่น **Job A** ส่งผลให้ **Job B)** ต้องเขียนไฟล์ลงดิสก์และรอให้อ่านขึ้นมาใหม่ ทำให้เสียเวลาในการ **I/O** สูงมาก
- ขาดความยืดหยุ่น: การเขียนโปรแกรมด้วย **MapReduce** ดั้งเดิมมีความซับซ้อน ต้องเขียนโค้ดภาษายาวเหยียดแม้จะเป็นงานง่าย ๆ

ตอนที่ **5:** วิวัฒนาการหลังยุค **MapReduce**

เพราะ **MapReduce** ช้าเกินไปในหลายกรณี ยุคต่อมาจึงเกิดเครื่องมือประมวลผลแบบแบตช์รุ่นใหม่ ๆ ที่เรียกว่า **Dataflow Engines (**เช่น **Apache Spark, Tez** และ **Flink)**

หลักการพัฒนาของเครื่องมือเหล่านี้คือ:

- ประมวลผลในหน่วยความจำ (**In-memory):** แทนที่จะเขียนพักลงดิสก์ทุกขั้นตอน ข้อมูลจะถูกส่งต่อกันผ่านหน่วยความจำหรือเครือข่ายโดยตรง ทำให้เร็วกว่าเดิมหลายเท่า
- มองภาพรวมเป็นกราฟ (**DAG - Directed Acyclic Graph):** แทนที่จะบังคับเป็นขั้นตอน **Map-then-Reduce** ตายตัว ระบบยุคใหม่จะมองภาพรวมเป็นแผนผังการไหลของข้อมูล แล้วช่วยปรับแต่งเส้นทางคำนวณให้มีประสิทธิภาพสูงสุดโดยอัตโนมัติ
- มีภาษาที่ใช้งานง่ายขึ้น: เช่น **Spark SQL** ที่ช่วยให้เราเขียนคำสั่งจัดการข้อมูลแบตช์ได้ง่ายเหมือนคลังข้อมูลทั่วไป

ตอนที่ **6:** สรุปท้ายบท

มาทบทวนประเด็นสำคัญของบทนี้กันครับ

หนึ่ง การประมวลผลแบบแบตช์ (**Batch Processing)** คือหัวใจสำคัญในการจัดการข้อมูลก้อนใหญ่ที่ไม่เร่งด่วน โดยยึดหลักข้อมูลต้องนิ่งและไม่ถูกแก้ไขระหว่างประมวลผล

สอง ปรัชญาจาก **Unix Tools (**การใช้ท่อเชื่อมโยงข้อมูลขนาดเล็ก) เป็นรากฐานทางความคิดที่ถูกนำมาขยายสเกลในระบบกระจาย

สาม **MapReduce** และ **HDFS** ช่วยให้เราประมวลผลข้อมูลระดับเพตาไบต์บนคลัสเตอร์ราคาถูกได้อย่างทนทานต่อความพัง แต่มีข้อเสียเรื่องความช้าจากการเขียนลงดิสก์บ่อยเกินไป

สี่ เครื่องมือยุคใหม่อย่าง **Dataflow Engines (**เช่น **Spark)** เข้ามาแก้ปัญหาความช้าด้วยการประมวลผลในหน่วยความจำและจัดการเส้นทางข้อมูลแบบกราฟ (**DAG)**

บทต่อไปคือบทที่สิบเอ็ด ซึ่งเป็นบทสุดท้ายของเนื้อหาในเล่ม ว่าด้วยเรื่องการมองภาพรวมอนาคตของระบบข้อมูลทั้งหมดครับ ขอบคุณที่ฟังครับ 😊